

Sistemas Operativos 2026

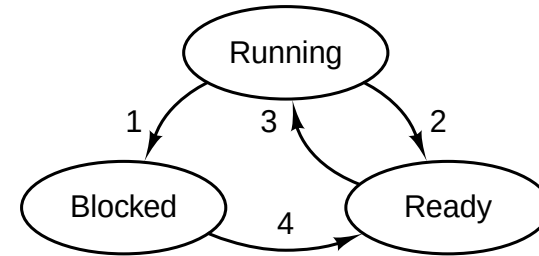


Clase 6 U2 Planificación

Ing. Juan Manuel Calvo

Planificación

- Suponga que hay varios procesos en el estado Ready
- ¿Cual se va a pasar a Running?
 - Hay muchas maneras de tomar esta decision



Planificación

- El planificador (scheduler) elige el próximo proceso a mover de de listo a corriendo.
 - ¿Como lo elige?
 - Usa uno de los algoritmos de planificación (scheduling).
 - Hasta ahora lo ignoramos
 - Las aplicaciones no deben depender del comportamiento del planificador.

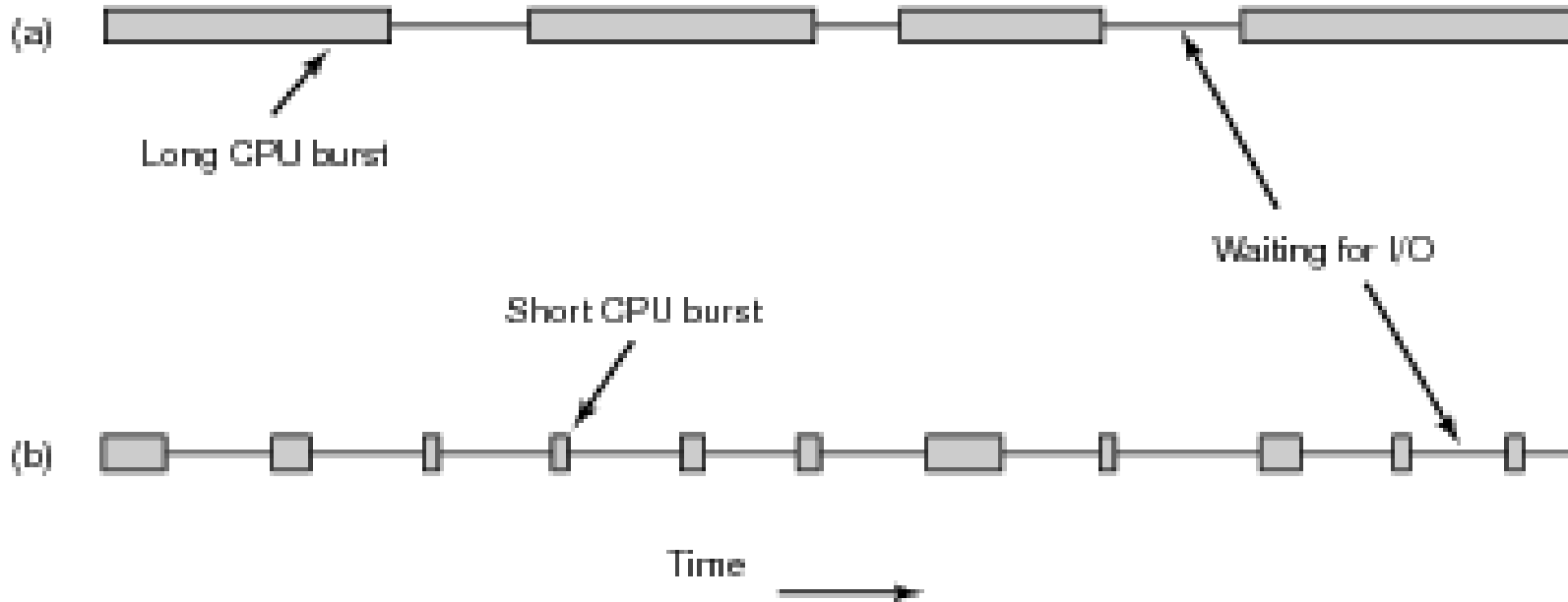
¿Qué se planifica?

- Se planifican tareas (jobs):
 - Procesos
 - hilos

Comportamiento de los procesos

- Los procesos alternan el uso de la CPU con las solicitudes de E/S.
- Las solicitudes de E/S se bloquean con frecuencia, ya sea esperando la entrada o cuando se ha escrito demasiado y no hay espacio de búfer disponible.
- Los procesos limitados por la CPU (**CPU-Bound**) son los que hacen cálculos más de lo que leen o escriben.
- Los procesos limitados por la E/S (**I/O-Bound**) gastan más tiempo en operaciones de E/S que haciendo cálculos en la CPU
- La velocidad absoluta de la CPU y los dispositivos de E/S es irrelevante; lo que importa es la proporción.
- En las últimas décadas las CPU se han vuelto mucho más rápidas en relación con los discos.

CPU bound & I/O bound



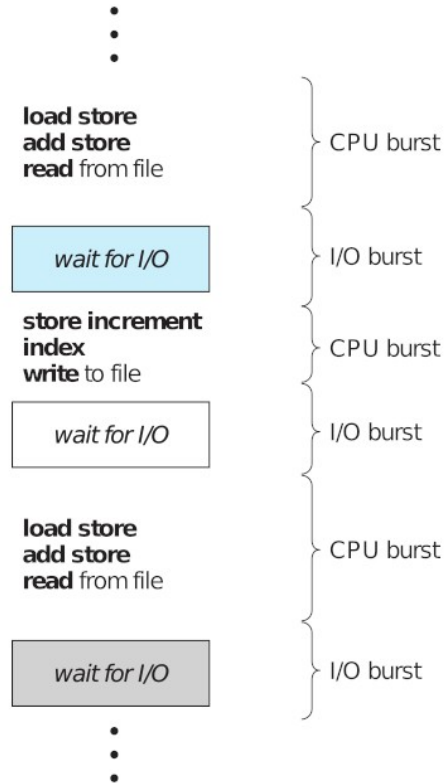
Compromisos del planificador

- Eficiencia
 - que gaste la mayor parte del tiempo ejecutando procesos de usuario,
 - ¿en qué otra cosa podría gastar el tiempo la CPU?
- Equidad - imparcialidad
 - Evitar la inanición, dar a cada proceso una "justa parte"
- Manejo de prioridades
 - Permitir a los procesos más "importantes" un mejor servicio.
- Restricciones de tiempo real
 - Un nivel de servicio garantizado
- Limitaciones de hardware
 - ¿cuántas instrucciones se usan para cambiar de proceso?

Planificación de la CPU

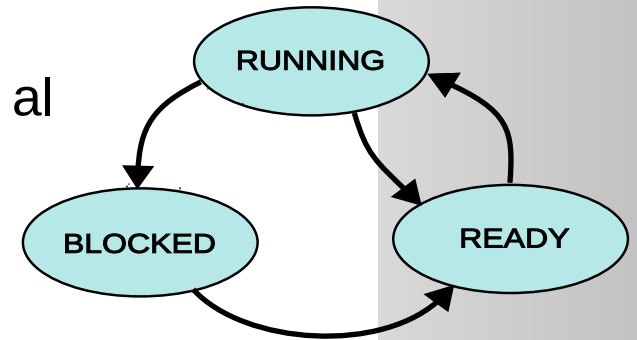
- Busca repartir los ciclos de la CPU a los procesos/hilos para lograr "buen rendimiento".
 - lo que sea que esto signifique.
- ¿Cual es el "perfil" de la aplicación?
 - Un programa alterna entre uso de la CPU y entrada/salida.
 - La pregunta pertinente para la planificación:
 - el programa es dominado por la computación en la CPU (compute-bound)
 - o dominado por la espera de entrada/salida (I/O bound).

Secuencia de ejecución de un programa



En que momentos se ejecuta la planificación:

- 1) Cuando un proceso cambia del estado de ejecución al estado bloqueado (por ejemplo, como resultado de una solicitud de E/S o una invocación de `wait()` para la terminación de un proceso secundario)
- 2) Cuando se produce una interrupción y un proceso cambia del estado de ejecución al estado listo
- 3) Cuando un proceso cambia del estado bloqueado al estado listo (por ejemplo, al finalizar la E/S)
- 4) Cuando un proceso finaliza



Tipos de algoritmos de planificación



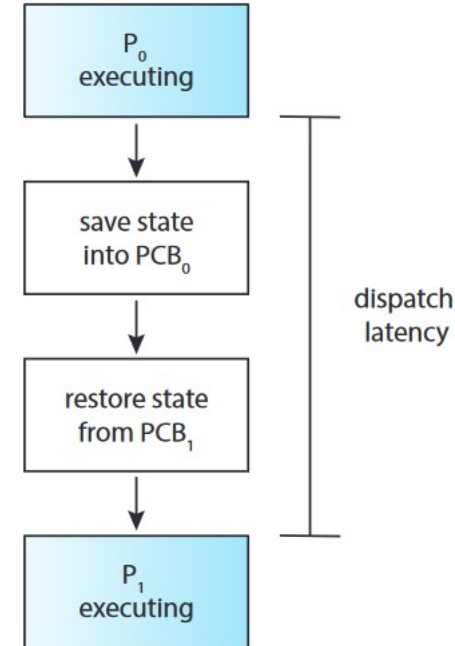
- **No apropiativo (nonpreemptive)**
 - Permite que un proceso se ejecute tanto tiempo como desee
 - Sólo conmuta cuando se bloquea
- **Apropiativo (preemptive)**
 - Elige un proceso y lo deja correr durante un tiempo fijo como máximo.
 - Al final del intervalo de tiempo si aun se sigue ejecutando, lo suspende y elige otro proceso para correr.
 - Requiere que se produzca una interrupción del reloj al final del intervalo de tiempo para devolver el control de la CPU al planificador.

Despachador

Es el módulo que transfiere el control al proceso elegido por el planificador:

Tareas:

- Cambiar el contexto de un proceso actual por el del elegido
- Cambiar a modo usuario
- Saltar a la ubicación correcta para continuar la ejecución del programa



Categorías

En general los algoritmos de planificación se pueden categorizar en:

- Por lotes (Batch)
- Interactivo
- Tiempo real



Metas de los algoritmos de planificación.



¿Qué estamos tratando de optimizar?

- **La utilización de la CPU:** fracción del tiempo que la CPU se está usando.
- **Rendimiento (Throughput):** promedio de tareas completadas por unidad de tiempo.
- **Tiempo de retorno (Turnaround time):** tiempo promedio entre la entrada y la terminación de la tarea.
- **Tiempo de espera (Waiting time):** tiempo promedio que el proceso está listo esperando.
- **Tiempo de respuesta:** tiempo hasta el que sistema responde a un comando.

Metas a optimizar

- **Todos los sistemas**
 - **Equidad:** dar a cada proceso una parte equitativa de la CPU.
 - **Aplicación de las políticas:** ver que la política establecida se lleve a cabo.
 - **Equilibrio:** mantener ocupadas todas las partes del sistema.

Metas a optimizar: Sistemas por lotes



- **Rendimiento:** maximice los trabajos por hora
- **Tiempo de respuesta:** minimice el tiempo entre el envío y la finalización
 - A veces se requiere que las tareas cortas se ejecuten inmediatamente.
- **Utilización de la CPU:** mantenga la CPU ocupada todo el tiempo

Metas a optimizar: Sistemas interactivos



- **Tiempo de respuesta:** responda a las solicitudes rápidamente
- **Proporcionalidad:** satisfaga las expectativas de los usuarios
 - Los usuarios tienen una percepción de las solicitudes "baratas" y "caras".
 - Los usuarios se sienten más satisfechos si las solicitudes "baratas" se completan rápidamente.
 - "Barato" y "caro" no siempre se corresponden con la realidad.

Metas a optimizar: sistemas en tiempo real



- **Cumplir con los plazos:** evitar la pérdida de datos (¡o algo peor!).
- **Previsibilidad:** los usuarios deben saber cuándo finalizarán sus solicitudes.
 - Requiere una ingeniería cuidadosa para adecuar las prioridades a los tiempos de finalización reales y los recursos disponibles.

Sobrecarga producida por la planificación

- Los cambios de contexto implican la ejecución de muchas instrucciones
- para optimizar:
 - Se deben minimizar la cantidad de cambios de contexto.
 - Tener en cuenta las estructuras de datos que usa el planificador.

Planificación de hilos

- Hilos en el espacio del usuario: la biblioteca de hilos es la que se encarga de la planificación , el kernel no interviene.
- Hilos en el kernel:
 - se planifican de la misma forma que los procesos
 - Suele ser beneficioso programar los hilos del mismo proceso consecutivamente, evitando cambios en el mapa de memoria.
 - Una lógica similar sugiere mantener los hilos del mismo proceso en una sola CPU, si es posible.
 - El planificador de hilos a nivel de aplicación puede gestionar las prioridades con mayor facilidad, aunque tambien se puede establecer las prioridades a nivel de kernel .

Algoritmos de planificación de sistemas de lotes (batch systems):

- **FCFS** - First-Come, First-Served
- **SJF** - Shortest Job First
- **SRTN** - Shortest Remaining Time Next

FCFS - First-Come, First-Served

- No-prioritario
- La cola de listos es una cola FIFO
- Las tareas que llegan se colocan al final de la cola.
- El despachador selecciona la primera tarea en la cola y esta corre hasta completar la ráfaga de CPU.
- Ventajas: simple, baja sobrecarga
- Desventajas: no es adecuado para sistemas interactivos, es posible grandes fluctuaciones en el tiempo promedio de retorno.

Ejemplo de FCFS

- Carga de trabajo de ejemplo con tres tareas:
 - Tarea 1: 24 unidades de tiempo, tarea 2: 3 unidades, tarea 3: 3 unidades.

- Plan FCFS

	tarea 1	tarea 2	tarea 3
0			
30	24	27	

- Tiempo total de espera: $0 + 24 + 27 = 51$
- Tiempo de espera promedio: $51/3 = 17$
- Tiempo total de retorno: $24 + 27 + 30 = 81$
- Tiempo promedio de retorno: $81/3 = 27$

SJF - Shortest Job First

- No-prioritario
- La cola de listos se trata como una cola de prioridades basada en los menores requerimientos de tiempo de CPU.
 - Las tareas que llegan se colocan en la posición correcta en la cola.
 - El despachador selecciona la tarea más corta (primera en la cola) y corre hasta completarla.
- Ventajas:
 - probablemente óptimo promedio tiempo de retorno.
- Desventajas:
 - Posible inanición.
 - ¡En la realidad no es posible implementarlo!
 - Solo se puede hacer aproximadamente: usar promedio exponencial de el comportamiento en el pasado para tratar de predecir la longitud de la próxima ráfaga de CPU.
 - Tomar la tarea cuya ráfaga prevista sea la más corta

Ejemplo de SJF

- La misma carga de 3 trabajos:
 - Tarea 1: 24 unidades de tiempo, tarea 2: 3 unidades, tarea 3: 3 unidades.

- Plan SJF

	tarea 2		tarea 3			tarea 1	
0		3		6			30

- Tiempo total de espera: $6 + 0 + 3 = 9$
- Tiempo promedio de espera: 3
- Tiempo total de retorno: $30 + 3 + 6 = 39$
- Tiempo promedio de retorno: $39/3 = 13$
- SJF siempre da mínimo tiempo de espera y de retorno.

Promediado exponencial

- $\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n$
- τ_{n+1} : largo previsto de la próxima ráfaga de CPU.
- t_n : longitud real de la última ráfaga de CPU.
- τ_n : predicción previa.
- $\alpha = 0$ implica no usar la historia reciente ($\tau_{n+1} = \tau_n$)
- $\alpha = 1$ implica $\tau_{n+1} = t_n$ (la predicción previa no se usa)
- $\alpha = 1/2$ implica ponderación (las ráfagas anteriores tienen menos y menos peso)

SRTN - Shortest Remaining Time Next

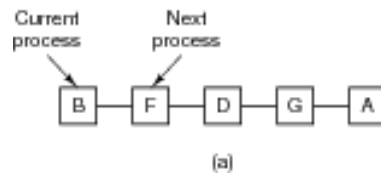
- Una versión preventiva del trabajo más corto primero
- El planificador elige el proceso cuyo tiempo de ejecución restante se estima más corto.
- Cuando arranca una tarea nueva su tiempo de ejecución estimado es comparado con el restante del proceso actual, si es menor este se suspende y la nueva tarea toma el control.
 - Este esquema permite que las tareas cortas obtengan un buen servicio.
 - Puede tener un problema con los adelantamientos permanentes.

Algoritmos de planificación de sistemas interactivos

- RR - Round Robin
- Prioridad
 - HPF Highest Priority First
 - Realimentación multi nivel (FB Multi-level feedback)
- Shortest Process Next
- Planificación garantizada
 - Linux CFS (Completely Fair Scheduling)
- Planificación por lotería
- Planificación de reparto justo (Fair-Share Scheduling)

RR - Round Robin

- Versión apropiativa del FCFS
- Los procesos se turnan para usar la CPU
- Se le asigna a cada proceso un intervalo (quantum) de tiempo a cada proceso, si el proceso sigue ejecutándose al final del cuanto, este es interrumpido
- Al sacar un proceso de la CPU este se coloca al final de la cola



Ejemplo de Round Robin

- La misma carga de 3 trabajos:
 - Tarea 1: 24 unidades de tiempo, tarea 2: 3 unidades, tarea 3: 3 unidades.
- Plan RR con un cuanto de tiempo de 3 unidades

Tarea 1	Tarea 2	tarea 3	tarea 1	tarea 1		tarea 1
0	3	6	9	12	15,18,21 ...	30

- Tiempo total de espera: $6 + 3 + 6 = 15$
- Tiempo promedio de espera: 5
- Tiempo total de retorno: $30 + 6 + 9 = 45$
- Tiempo promedio de retorno:
- RR da tiempos de espera y retorno intermedios entre FCFS y SJF.

Propiedades del Round Robin

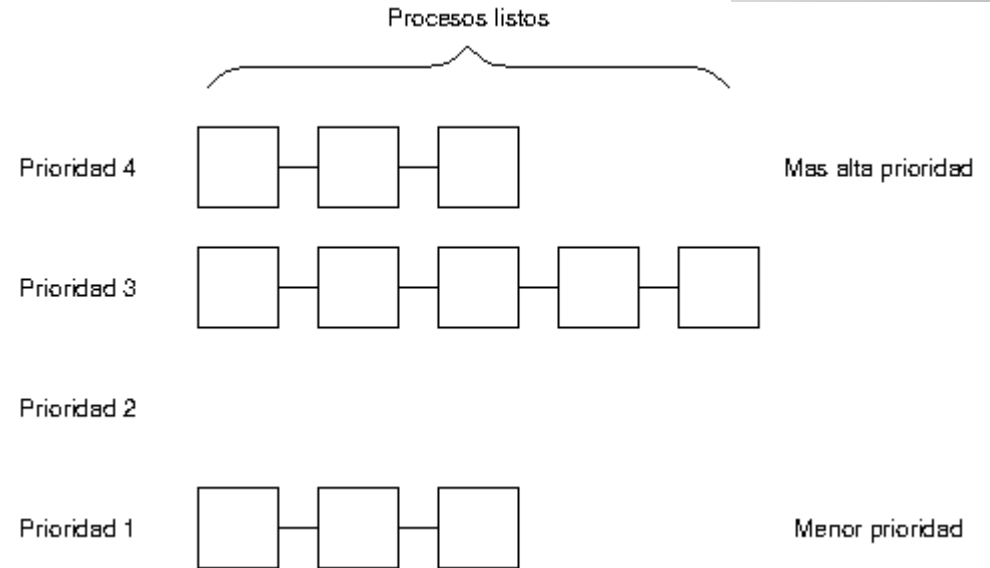
- Ventajas: simple, funciona para sistemas interactivos.
- Desventajas: Si el cuanto es muy chico, se gasta mucho tiempo en cambios de contexto; si es demasiado largo (i. e. más largo que la ráfaga media de CPU), se aproxima al FCFS.
- Valores típicos: algunos mseg.
- Regla: elija el cuanto suficientemente grande para que la mayoría de las tareas terminen su ráfaga de CPU en un cuanto.
- RR supone que todos los procesos tienen la misma importancia.

Importancia que un buen factor de cuanto

- Los cambios de contexto son onerosos
 - centenares de instrucciones de CPU
 - si el cuanto es demasiado pequeño, mucho tiempo de CPU se gastará en sobrecarga de cambio de contexto.
 - ... y la máquina parecerá más lenta de lo que realmente es.
- La interactividad requiere un rápido tiempo de respuesta.
 - Los procesos interactivos necesitan servicio rápido, especialmente después de que ocurre entrada/salida.
 - Si el cuanto es demasiado largo, el tiempo de espera se incrementa.
 - ... y la máquina parecerá mas lenta de lo que realmente es.
- El cuanto debe ser $\gg$ que el costo del cambio de contexto, pero $\ll$ tiempo de respuesta deseado.

Planificación de prioridad

- No todos los procesos son igualmente importantes
- Asignarles niveles de prioridad
- Siempre se eligen los procesos de la cola de prioridad más alta
- Si no hay procesos en una prioridad se continúa con la siguiente



Ajustes de prioridad

- Reducir periódicamente la prioridad del proceso en ejecución.
- Eventualmente, cae por debajo de la prioridad del siguiente proceso.
- Alternativa: aumentar la prioridad de los procesos que no se están ejecutando.
- Denominado envejecimiento del proceso (process aging).
- O usar ambas opciones.

Ajuste dinámico de prioridad

- Ajuste la prioridad del proceso según su historial reciente.
- Ejemplo: aumente la prioridad de los procesos que no se están ejecutando; disminuya la prioridad de los procesos en ejecución, como se indicó anteriormente.
- Aumente la prioridad de los procesos vinculados a E/S:
 - Si el proceso utilizó $1/f$ de su último quantum, aumente su prioridad proporcional a f .
- Utilice clases de prioridad: tenga colas separadas para cada nivel de prioridad y ejecute cada cola en turnos rotatorios; cambie a la cola de menor prioridad cuando esta esté vacía.

Colas de ejecución (Run Queues)

- Cada cola de ejecución es una lista enlazada.
- Para aumentar o disminuir la prioridad de un proceso, muévelo a una lista diferente.
- Dos esquemas para el envejecimiento de la prioridad:
 - Con pocos procesos: tener un contador detallado para cada proceso, que se incrementa con una interrupción de reloj; en cierto límite, aumentar la prioridad.
 - Con muchos procesos y colas: mover periódicamente cada lista al siguiente nivel superior.

Quantum variable

- Hay muchos procesos requieren poco tiempo de CPU.
- Dado que los cambios de proceso pueden ser costosos, no los realice con demasiada frecuencia.
- Solución a ambos problemas: asignar cuantos más largos a las colas de menor prioridad.
- Cola de máxima prioridad: un cuanto.
- Segunda cola: asignación de CPU para dos cuantos.
- Tercera cola: cuatro cuantos, etc.
- Solución alternativa: un cuanto "corto" (inicial) con alta prioridad y, posteriormente, un cuanto "largo" con baja prioridad.

Ayudando a los procesos interactivos

- Buscar señales de entrada del usuario
 - Cuando se produzcan, dar al proceso una prioridad muy alta
 - Ejemplo: en CTSS, cuando un usuario pulsaba un retorno de carro (la tecla ENTER), el proceso tenía máxima prioridad
- Hoy en día, algo más difícil de hacer:
 - ¿qué se considera "interactivo" en un proceso en red?
 - En el caso de un movimiento del mouse, ¿a qué proceso se atribuye?

Prioridades de los procesos

- ¿Qué procesos deberían tener mayor prioridad?
- Problemas administrativos, ¿como priorizamos?
- Rendimiento del sistema
 - Procesos del kernel (hasta cierto punto)
 - Procesos de servicios interactivos (p. ej., servidor X)
- Los usuarios pueden reducir la prioridad de sus propios procesos, a veces para evitar competir consigo mismos.

Prioridades en Unix

- Convención: los números más bajos indican prioridades más altas
- El valor "Nice" es un modificador especificado por el usuario
- Un valor nice de +20 especifica un proceso con una prioridad muy baja
- Solo el usuario root puede establecer un nice negativo
- El valor predeterminado es 0
- Nota: Esto es en la API; la métrica interna puede ser diferente

Prioridades en Linux

```
$ ps -eo user,pri,ni,sz,comm
```

USER	PRI	NI	SZ	COMMAND
root	19	0	10606	systemd
root	19	0	0	kthreadd
root	39	-20	0	kworker/R-rcu_gp
root	139	-	0	migration/0
root	39	-20	0	kworker/R-inet_frag_wq
root	90	-	0	watchdogd
root	19	0	856	bluetoothd
root	19	0	588	cron
nobody	19	0	377	thd
jmc	19	0	1075	syslog_server
www-data	19	0	1168	nginx
postfix	19	0	2722	qmgr
jmc	19	0	871	ps

Los valores PRI toman en cuenta la niceness y la prioridad dinámica del proceso.



UNIVERSIDAD
DEL CEMA

UCEMA

Planificación garantizada

- Para n usuarios o procesos en un sistema, asignar a cada uno $1/n$ de la CPU.
- Medir el uso real de la CPU.
- Calcular el tiempo de CPU asignado al proceso.
- Observar la relación entre ambos: 0,5 significa que solo ha tenido la mitad de la CPU a la que tiene derecho, por lo que tiene prioridad sobre un proceso con una relación de 2,0.

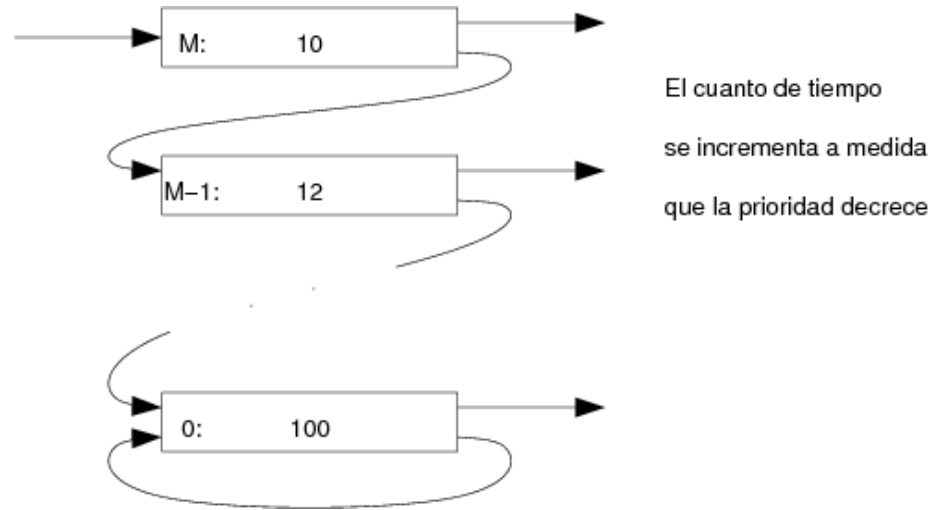
Planificación por lotería

- Dar boletos de lotería a cada proceso.
- Los procesos de mayor prioridad obtienen más boletos; los de menor prioridad, menos.
- Al planificar, elegir un boleto al azar.
- El proceso que contiene ese boleto es el que se ejecuta.
- Nota: Los procesos pueden intercambiar boletos, por ejemplo, entre el cliente y el servidor.

Planificación de reparto equitativo

- En algunos sistemas, no compiten los procesos sino los usuarios.
- No queremos fomentar la división de los procesos solo para obtener una mayor cuota de CPU.
- Solución: tomar decisiones basadas en el consumo de CPU del usuario en lugar del consumo de CPU del proceso.
- Se siguen aplicando las prioridades.

Realimentación multi nivel (FB Multi-level feedback)



- Cada nivel de prioridad tiene asignados una cola de listos y un cuanto de tiempo.
- Los procesos entran inicialmente en la cola de más alta prioridad, y después bajan con cada interrupción del timer (son penalizados por usar mucho tiempo de CPU)
- La cola inferior es una standard round robin
- Los procesos en una dada cola no son planificados hasta que todas las colas superiores estén vacías.

Planificación en tiempo real

- Deben responder en el instante justo
- Un proceso tardío puede ser un proceso inútil
- Dos tipos: hard y soft
- Los sistemas de tiempo real hard tienen plazos que deben cumplirse; se utilizan para control de procesos, aviónica, etc.
- El tiempo real soft se esfuerza al máximo, pero puede incumplir plazos ocasionalmente.
- Ambos dependen del conocimiento del tiempo de procesamiento de cada solicitud.

Eventos periódicos

- Muchos eventos en tiempo real ocurren con una frecuencia regular.
- Supongamos que hay m eventos, donde el evento i requiere C_i segundos de CPU y ocurre cada P_i segundos.
- El sistema funciona si y solo si

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq 1$$

- No olvide incluir los tiempos de conmutación de proceso.

Eventos aperiódicos

- Otros eventos no ocurren con regularidad.
- La restricción básica es la misma: la carga total debe ser menor que la capacidad del sistema.
- Diseñar tales sistemas es más difícil.

Planificación de varios procesos de tiempo real



- Ejemplo
 - Tarea T1 con período 10 y tiempo de CPU 3
 - Tarea T2 con período 10 y tiempo de CPU 1
 - Tarea T3 con período 15 y tiempo de CPU 8
- Planificación posible: se repite cada 30 segundos
 - T1 desde 0 a 3, 12 a 15, 24 a 27
 - T2 desde 3 a 4, 15 a 16, 27 a 28
 - T3 desde 4 a 12, 16 a 24
- Si T2 tuviera un período de 5 (en lugar de 10) entonces no es posible la planificacion.
- Prueba simple:
 - La tarea T_i necesita usar la CPU por C_i/P_i fracción de unidad.
 - Utilización = suma de los C_i/P_i
 - El conjunto de tareas es planificable si y solo si la utilización es 1 o menos

Combinación de procesos comunes y en tiempo real

- Algunos sistemas tienen ambos tipos de procesos (y planificadores).
- Estrategia general: priorizar los procesos en tiempo real sobre todos los procesos convencionales.
- Planificarlos round-robin o posiblemente incluso de forma no preventiva.

Programación y sistemas multiprocesador

- ¿Qué procesos se ejecutan en qué procesador?
- ¿Tiene importancia?
- ¿Cuáles son las implicaciones?



Multiprocesamiento simétrico

- Cada procesador puede ejecutar el kernel o procesos de usuario.
- Es posible tener más de una CPU en el kernel simultáneamente
- Requiere un bloqueo (Locking) de grano fino; un único "bloqueo grande" es casi lo mismo que un multiprocesador asimétrico
- El bloqueo es un tema muy importante para los multiprocesadores

Afinidad del procesador

- A veces, es mejor (o necesario) que un proceso determinado se ejecute en una CPU específica.
- Ejemplo citado anteriormente: mapa de memoria (y caché).
- Problemas de E/S: a veces, un dispositivo de E/S específico está en un bus local.

¿Cómo medimos el tiempo de la CPU?

- Los primeros relojes eran de baja resolución.
 - Clásico de Unix: 60 Hz.
 - Demasiado burdo para medir una fracción de un quantum.
 - Además, el reloj era un dispositivo de E/S, por lo que su acceso era lento.

Tiempo estadístico

- En cada tic del reloj, suma 1 al contador de CPU del proceso actual.
- En realidad, dos contadores, uno para el modo de usuario y otro para el modo kernel.
- No es preciso en períodos cortos:
 - Un proceso puede ejecutarse durante un tiempo demasiado corto y no cargarse.
 - Un proceso puede comenzar su quantum justo antes del tic del temporizador y cargarse demasiado.
- Sin embargo, estadísticamente es suficientemente bueno.

E/S y memoria

- El tiempo de CPU no es el único recurso escaso.
- Especialmente hoy en día, el rendimiento total del sistema está limitado por el ancho de banda de E/S y la disponibilidad de memoria.
- Debe leer los programas desde el disco.
- Debe tener memoria para ellos.
- Eso puede significar paginar otro proceso, lo que aumenta la carga en el disco.
- El planificador debe interactuar con el subsistema de E/S y el subsistema de memoria.

Algoritmos de planificación

- Los sistemas modernos pueden tener muchos procesos en ejecución.
- Incluso en las CPU modernas, no queremos algoritmos de planificación que iteren sobre todos los procesos.
- Linux utiliza un programador $O(1)$: las decisiones de programación toman un tiempo constante, independientemente del número de procesos.

Política versus Mecanismo

- Incluir los mecanismos básicos en el kernel
- Permitir que los procesos de usuario establezcan parámetros que controlen la planificación
- El ejemplo más simple: el comando `nice`



Planificador de Linux

- Tiempo real y prioridades
- La programación en tiempo real puede ser por turnos o FCFS
- Computación dinámica de los cuantos de tiempo
- Es posible la interrupción del kernel si este no tiene bloqueos

Linux CFS

- CFS Scheduler docs.kernel.org
- Inside the Linux 2.6 Completely Fair Scheduler



Planificación en Windows I

- Planificación en tiempo real y por prioridades
- Clases de prioridad: Tiempo real, Alta, Superior a lo normal, Normal, Inferior a lo normal, Inactiva
- Prioridad relativa dentro de la clase: Crítica en el tiempo, Máxima, Superior a lo normal, Normal, Inferior a lo normal, Inactiva
- Prioridad efectiva calculada a partir de esta matriz
- Si consume CPU hasta expirar el cuanto se reduce la prioridad.
- Adicionalmente se aumenta la prioridad para el proceso asociado con la ventana actual
 - Unix no puede hacer eso fácilmente; el gestor de ventanas no conoce los procesos

Planificación en Windows II

- Windows Internals Chapter 4 Thread scheduling pag 311
 - The Windows scheduling code is implemented in the kernel. There's no single "scheduler" module or routine, however. The code is spread throughout the kernel in which scheduling related events occur. The routines that perform these duties are collectively called the kernel's dispatcher.
 - Windows schedules at the thread granularity level. This approach makes sense when you consider that processes don't run; rather, they only provide resources and a context in which their threads run. Because scheduling decisions are made strictly on a thread basis, no consideration is given to what process the thread belongs to.
- Ver también [Microsoft Learn Scheduling](#)